Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Khoa học Tự nhiên
Khoa Công nghệ Thông tin



BÁO CÁO ĐỒ ÁN 1
Bloxorz Solver

Môn học: CSC14003 – Cơ sở trí tuệ nhân tạo

Lớp: 24TN – **Nhóm 04**

Sinh viên thực hiện

Lê Minh Trí (24120015)
Võ Đình Trường (24120234)
Hồ Ngọc Huy (24120009)
Bùi Anh Hào (24120305)

Giảng viên hướng dẫn

GV. Lê Hoài Bắc
GV. Nguyễn Ngọc Đức

Học kỳ I, năm học 2026–2027
Thành phố Hồ Chí Minh, 1 tháng 11 năm 2026

Video demo

Mục lục

Bảng đóng góp iii

1 Mô hình hóa bài toán 1

1.1 Phát biểu bài toán và quy tắc Bloxorz . . . 1

1.2 Biểu diễn không gian trạng thái . . . 1

1.2.1 Khối nguyên . . . 2

1.2.2 Khối tách và mặt nạ trạng thái cầu . . . 2

1.3 Hành động và chuyển trạng thái . . . 3

1.4 Điều kiện hợp lệ và các cơ chế nâng cao . . . 4

1.4.1 Điều kiện nâng đỡ, Fragile và Goal . . . 4

1.4.2 Công tắc và cầu . . . 4

1.4.3 Tách và ghép khối . . . 4

1.5 Trạng thái ban đầu và điều kiện đích . . . 5

1.6 Mô hình tìm kiếm trên đồ thị và hàm chi phí . . . 5

2 Thiết kế và hiện thực hệ thống 7

2.1 Kiến trúc tổng thể . . . 7

2.2 Hiện thực mô hình và GameEngine . . . 8

2.3 Kiến trúc hệ thống giải tự động . . . 9

2.4 Giao diện đồ họa và điều khiển . . . 9

2.5 Đo hiệu năng và xác minh kết quả . . . 10

3 Các thuật toán tìm kiếm 10

3.1 Breadth-First Search . . . 11

3.2 Depth-First Search . . . 11

3.3 Uniform-Cost Search . . . 12

3.4 A* Search . . . 13

3.4.1 Heuristic cho khối nguyên . . . 14

3.4.2 Heuristic cho hai khối đơn vị . . . 14

4 Thiết kế thực nghiệm 16

4.1 Câu hỏi đánh giá và bộ màn chơi 16
4.2 Môi trường thực nghiệm 16
4.3 Quy trình và kiểm soát công bằng 16
4.4 Chỉ số đánh giá 17
4.5 Phương pháp xác minh lời giải 17

5 Kết quả và thảo luận **17**
5.1 Bảng kết quả tổng quan 17
5.2 Biểu đồ theo màn chơi 18
5.3 Chất lượng lời giải và đánh đổi 18

6 Kết luận và hướng phát triển **20**
6.1 Chức năng và kết quả đạt được 20
6.2 Giới hạn của kết luận 21
6.3 Hướng phát triển 21

Tài liệu tham khảo **22**

A Khai báo sử dụng AI **23**

B Kết quả thực nghiệm đầy đủ **24**

Bảng đóng góp

Bảng 1: Công việc và sản phẩm đóng góp của các thành viên

Thành viên	Công việc và sản phẩm đã thực hiện	Mức độ hoàn thành
Lê Minh Trí 24120015	• Thiết kế kiến trúc hệ thống và quy ước dữ liệu dùng chung cho các bộ giải. • Xây dựng `SolverManager`, bộ xác minh lời giải và kiểm thử tích hợp. • Triển khai công cụ chạy thực nghiệm; xuất CSV, bảng tổng hợp và biểu đồ. • Thiết kế và xây dựng các màn chơi nâng cao 11–15. • Biên soạn các phần tổng quan, thiết kế hệ thống, kết quả và thảo luận; tích hợp báo cáo bằng LaTeX.	100%
Bùi Anh Hào 24120305	• Hiện thực A* và heuristic cho khối nguyên, khối tách và dịch chuyển tức thời. • Phát triển GUI 3D, bộ điều khiển, luồng nền và chức năng phát lại; xây dựng màn 08–10. • Viết kiểm thử cho A*, heuristic và tương tác giao diện. • Viết kịch bản và thực hiện video demo. • Trình bày thuật toán A*, thiết kế GUI/phát lại và phân tích thời gian cùng số nút mở rộng.	100%
Hồ Ngọc Huy 24120009	• Hiện thực Fragile, cầu, ba loại công tắc và cơ chế tách–ghép khối. • Xây dựng hàm chi phí 0/1/2, bộ giải UCS, bộ dựng hình và hoạt ảnh; xây dựng màn 05–07. • Viết kiểm thử cho luật nâng cao, hàm chi phí và UCS. • Trình bày cơ chế nâng cao, hàm chi phí, UCS và phân tích kết quả bộ nhớ.	100%
Võ Đình Trường 24120234	• Xây dựng mô hình trạng thái, bộ nạp và bộ kiểm tra dữ liệu màn chơi. • Hiện thực chuyển động cơ bản trong `GameEngine`, BFS và DFS; xây dựng màn 01–04. • Viết kiểm thử cho mô hình, bộ luật và hai bộ giải. • Trình bày mô hình hóa bài toán, mã giả BFS/DFS và phân tích kết quả của hai thuật toán.	100%

1 Mô hình hóa bài toán

1.1 Phát biểu bài toán và quy tắc Bloxorz

Trong Bloxorz, người chơi lăn khối hộp $1 \times 1 \times 2$ trên một bàn chơi hữu hạn để đưa khối từ ô Start tới ô Goal [1]. Màn chơi chỉ hoàn thành khi khối đứng thẳng trên Goal; khối nằm ngang hoặc nằm dọc qua Goal chưa thỏa điều kiện đích. Ngoài Floor và Void, phiên bản của nhóm hỗ trợ Fragile, Bridge, Soft Switch, Heavy Switch và Split Switch theo yêu cầu đề bài [2].

Trò chơi có tính xác định và quan sát đầy đủ: ở mỗi thời điểm, lưới và trạng thái hiện tại của các cầu đều được biết; từ một trạng thái và hành động hợp lệ, kết quả được xác định duy nhất. Ta tách dữ liệu thành hai phần:

- **Dữ liệu tĩnh của màn chơi**: lưới, Start, Goal, danh sách cầu, trạng thái mở ban đầu, công tắc và tác động của từng công tắc. Các dữ liệu này được xác định trước và không thay đổi trong khi tìm kiếm.
- **Trạng thái động**: tư thế/vị trí của khối và trạng thái mở–đóng của các cầu. Đây là phần phải nằm trong mỗi trạng thái tìm kiếm.

Gọi R, C lần lượt là số hàng và số cột. Một màn chơi được mô tả bởi

$$L = (\Gamma, p_0, g, \mathcal{B}, \mathcal{W}), \tag{1}$$

trong đó Γ là lưới $R \times C$, p_0 là Start, g là Goal, $\mathcal{B}$ là tập cầu và $\mathcal{W}$ là tập công tắc.

Bảng 2: Các loại ô cơ bản trong lưới của màn chơi

Ký hiệu	Loại ô	Ý nghĩa
`.`	Floor	Ô sàn thông thường
`#`	Void	Không có sàn; có thể được một cầu phủ lên
`S`	Start	Vị trí ban đầu của khối đứng
`G`	Goal	Đích; chỉ khối nguyên đứng mới hoàn thành màn chơi
`F`	Fragile	Không chịu được khối nguyên ở tư thế đứng

1.2 Biểu diễn không gian trạng thái

Một trạng thái đầy đủ được biểu diễn bởi

$$s = (b, m), \tag{2}$$

trong đó b mô tả khối và m là mặt nạ bit lưu trạng thái các cầu. Thành phần b phải ghi tư thế khi khối nguyên, hoặc chỉ số khối lập phương đang được điều khiển khi khối tách. Thiếu thông tin này, trạng thái không xác định được nước đi kế tiếp.

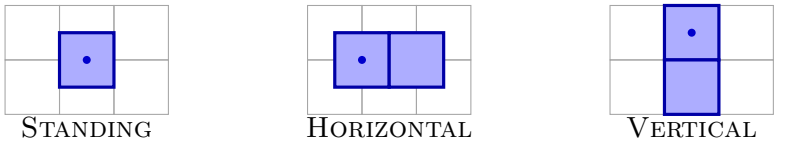
1.2.1 Khối nguyên

Khi chưa bị tách, khối có dạng

$$b = \text{Normal}(p, o), \qquad o \in \{\textsc{Standing}, \textsc{Horizontal}, \textsc{Vertical}\}, \tag{3}$$

với $p = (r, c)$ là ô mốc (anchor): ô duy nhất khi khối đứng, ô bên trái khi khối nằm ngang và ô phía trên khi khối nằm dọc. Tập ô mà khối chiếm là

$$\text{Occ}(p, o) = \begin{cases} \{(r, c)\}, & o = \textsc{Standing}, \\ \{(r, c), (r, c+1)\}, & o = \textsc{Horizontal}, \\ \{(r, c), (r+1, c)\}, & o = \textsc{Vertical}. \end{cases} \tag{4}$$



Standing $\{(r, c)\}$ | Horizontal $\{(r, c), (r, c+1)\}$ | Vertical $\{(r, c), (r+1, c)\}$

Hình 1: Ba tư thế của khối nhìn từ trên xuống; chấm đậm là ô mốc

Hình 1 cho thấy chỉ lưu tọa độ là chưa đủ: cùng một ô mốc nhưng tư thế khác nhau sẽ chiếm các ô khác nhau, dẫn đến tập hành động hợp lệ và trạng thái kế tiếp khác nhau.

1.2.2 Khối tách và mặt nạ trạng thái cầu

Trong bước xử lý Split Switch, thành phần khối trước hết trở thành

$$b = \text{Split}(p_a, p_b, i), \qquad i \in \{0, 1\}, \tag{5}$$

trong đó p_a, p_b là vị trí hai khối lập phương đơn vị và i xác định khối đang được điều khiển. Chỉ khối này di chuyển; hành động SwitchCube đổi i thành $1 - i$ mà không đổi vị trí.

Với k cầu, bit thứ j của m cho biết cầu j đang mở hay đóng:

$$\text{Open}(j, m) = ((m \gg j)\ \&\ 1) = 1. \tag{6}$$

Hai trạng thái có cùng vị trí và tư thế nhưng khác mặt nạ cầu là hai đỉnh khác nhau, vì một nước đi có thể hợp lệ ở trạng thái này nhưng làm khối rơi ở trạng thái kia. Tương tự, hai trạng thái khối tách chỉ khác khối đang điều khiển cũng phải được phân biệt.

1.3 Hành động và chuyển trạng thái

Tập hành động là

$$A = \{\textsc{Up}, \textsc{Down}, \textsc{Left}, \textsc{Right}, \textsc{SwitchCube}\}. \tag{7}$$

Bốn hành động di chuyển áp dụng cho cả hai dạng khối; SwitchCube chỉ hợp lệ khi khối đã tách. Với khối nguyên, hàm lăn phụ thuộc vào ô mốc hiện tại $p = (r, c)$, tư thế và hành động. Mỗi ô của Bảng 3 cho cặp (p', o') sau khi lăn: p' là ô mốc mới, còn o' nhận một trong ba giá trị S (Standing), H (Horizontal) hoặc V (Vertical).

Bảng 3: Ô mốc và tư thế sau khi lăn khối nguyên

Tư thế trước	**Up**	**Down**	**Left**	**Right**
Standing	$(r-2, c), V$	$(r+1, c), V$	$(r, c-2), H$	$(r, c+1), H$
Horizontal	$(r-1, c), H$	$(r+1, c), H$	$(r, c-1), S$	$(r, c+2), S$
Vertical	$(r-1, c), S$	$(r+2, c), S$	$(r, c-1), V$	$(r, c+1), V$

Khi khối tách, một hành động di chuyển cộng vectơ $(-1, 0)$, $(1, 0)$, $(0, -1)$ hoặc $(0, 1)$ vào vị trí khối đang được điều khiển. Khối này không được đi vào vị trí của khối còn lại; một khối lập phương riêng lẻ cũng không được đi vào Goal. Hai khối tự động ghép khi khoảng cách Manhattan giữa chúng bằng 1. Nếu kề theo hàng, khối mới có tư thế Horizontal và ô mốc là ô bên trái; nếu kề theo cột, tư thế là Vertical và ô mốc là ô phía trên.

Hàm chuyển trạng thái được viết dưới dạng

$$T : S \times A \to S \cup \{\textsc{Invalid}, \textsc{Fall}\}. \tag{8}$$

Hàm T gồm biến đổi hình học, kiểm tra điều kiện nâng đỡ, kích hoạt công tắc mới được nhấn, cập nhật trạng thái cầu và kiểm tra lại điều kiện nâng đỡ ở trạng thái cuối. Một nước đi rơi vào ô không được nâng đỡ không thể kích hoạt công tắc để "cứu" chính nó. Riêng SwitchCube chỉ đổi khối đang điều khiển, không thay đổi bàn chơi hay kích hoạt lại công tắc.

1.4 Điều kiện hợp lệ và các cơ chế nâng cao

1.4.1 Điều kiện nâng đỡ, Fragile và Goal

Một chuyển động chỉ hợp lệ khi mọi ô mà khối chiếm đều nằm trong bàn chơi và được nâng đỡ. Floor, Start và Goal có thể nâng đỡ khối; ô Void chỉ nâng đỡ được nếu có cầu đang mở. Khối nguyên đứng trên Fragile sẽ rơi, trong khi khối nằm hoặc khối lập phương riêng lẻ vẫn được nâng đỡ.

Bảng 4: Điều kiện nâng đỡ theo loại ô và dạng khối

Loại ô	**Khối đứng**	**Khối nằm**	**Khối đơn vị**
Floor/Start	Hợp lệ	Hợp lệ	Hợp lệ
Void hoặc cầu đóng	Rơi	Rơi	Rơi
Cầu mở	Hợp lệ	Hợp lệ	Hợp lệ
Fragile	Rơi	Hợp lệ	Hợp lệ
Goal	Có thể thắng	Được nâng đỡ, chưa thắng	Không được đi vào

1.4.2 Công tắc và cầu

Công tắc chỉ phát sinh hiệu ứng khi chuyển từ trạng thái chưa bị nhấn sang bị nhấn. Soft Switch được kích hoạt khi bất kỳ phần nào của khối nguyên hoặc một khối lập phương đè lên nó. Heavy Switch và Split Switch chỉ được kích hoạt khi khối nguyên đứng đúng trên công tắc.

Mỗi công tắc có thể tác động tới một hoặc nhiều cầu bằng ba phép toán: Toggle, SetOpen và SetClosed. Nếu nhiều hiệu ứng cùng tác động một cầu trong một bước chuyển, `GameEngine` xử lý số lần Toggle theo tính chẵn lẻ, sau đó áp dụng SetClosed và cuối cùng là SetOpen. Vì vậy, SetOpen được ưu tiên khi các hiệu ứng xung đột. Mặt nạ cầu mới trở thành một phần của trạng thái kế tiếp.

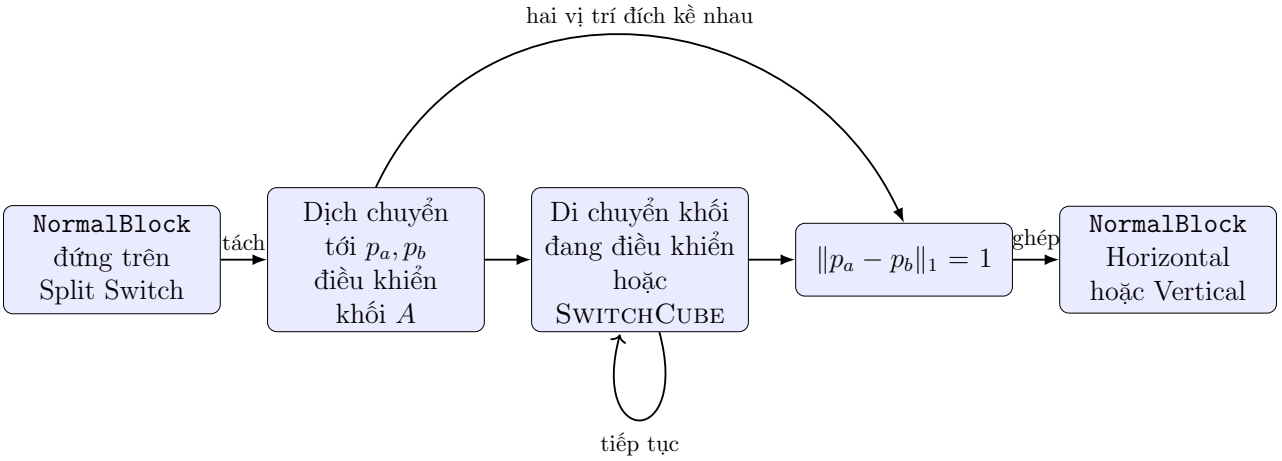
Bảng 5: Điều kiện kích hoạt các loại công tắc

Công tắc	**Khối đứng**	**Khối nằm**	**Khối đơn vị**	**Tác động chính**
Soft	Có	Có	Có	Thay đổi trạng thái cầu
Heavy	Có	Không	Không	Thay đổi trạng thái cầu
Split	Có	Không	Không	Tách và dịch chuyển

1.4.3 Tách và ghép khối

Split Switch tách khối và đưa hai khối đơn vị tới các vị trí `split_targets`; khối A được điều khiển trước. Mỗi khối đơn vị có thể kích hoạt Soft Switch nhưng không kích hoạt Heavy/Split Switch hoặc hoàn thành màn chơi. Nếu hai vị trí đích kề trực giao, `GameEngine` ghép khối ngay

trong hành động dịch chuyển tức thời. Trường hợp còn lại, người chơi có thể di chuyển hoặc đổi khối đang điều khiển để đưa hai khối tới vị trí kề nhau.



Hình 2: Vòng đời của khối khi kích hoạt Split Switch và ghép lại

1.5 Trạng thái ban đầu và điều kiện đích

Trạng thái đầu là $s_0 = (\text{Normal}(p_0, \textsc{Standing}), m_0)$, trong đó m_0 mã hóa các cầu mở ban đầu. Tập đích là

$$G = \{(b, m) \in S \mid b = \text{Normal}(g, \textsc{Standing})\}. \tag{9}$$

Nói cách khác, khối đang nằm trên Goal hoặc một khối đơn vị ở gần Goal đều không thỏa điều kiện kết thúc. Từ đây, g luôn là tọa độ ô Goal, còn G là tập các trạng thái đích.

1.6 Mô hình tìm kiếm trên đồ thị và hàm chi phí

Ký hiệu $\mathcal{G} = (V, E)$ dành riêng cho đồ thị tìm kiếm, phân biệt với tập đích G. Tập V gồm các trạng thái hợp lệ có thể đạt từ trạng thái đầu; tập cạnh được xác định bởi

$$E = \{(s, a, s') \mid T(s, a) = s',\ a \in A\}. \tag{10}$$

Chỉ bước chuyển hợp lệ hoặc thắng mới tạo cạnh cho bộ giải; hành động không hợp lệ và trạng thái rơi bị loại. Nếu $N = R \cdot C$ và có k cầu, một cận trên đơn giản của số trạng thái là

$$|V| \le 2^k \left(3N + 2N(N-1)\right), \tag{11}$$

gồm tối đa $3N$ cấu hình khối nguyên và $2N(N-1)$ cấu hình hai khối đơn vị có phân biệt khối đang điều khiển. Các điều kiện nâng đỡ và hình học làm số trạng thái thực tế nhỏ hơn nhiều,

nhưng cận này cho thấy mặt nạ cầu và trạng thái khối tách làm không gian tìm kiếm tăng nhanh.

Đồ thị có chu trình do lăn ngược, đổi trạng thái cầu và đổi khối điều khiển hai lần. Mỗi đỉnh phải chứa đầy đủ b và m để phân biệt các nước đi tiếp theo. Vì vậy, các thuật toán cần kiểm soát trạng thái lặp khi tìm kiếm trên đồ thị [3].

Hàm chi phí chuẩn $c : S \times A \times S \to \{0, 1, 2\}$ gán trọng số cho mỗi cạnh hợp lệ như Bảng 6. Chi phí một đường đi là tổng chi phí các cạnh; các thuật toán khác nhau có thể tối ưu những mục tiêu khác nhau.

Bảng 6: Hàm chi phí chuẩn dùng cho UCS và A*

Hành động	**Trường hợp**	**Chi phí**
SwitchCube	Chỉ đổi khối đang điều khiển	0
Di chuyển	Kích hoạt Split Switch; trạng thái sau gồm hai khối đơn vị	1
Di chuyển	Trạng thái sau chạm Fragile, ngoài trường hợp trên	2
Di chuyển	Trường hợp hợp lệ còn lại	1

Với khối nguyên, Fragile được xét ở các ô khối chiếm trong trạng thái sau; với khối đơn vị đang di chuyển, xét ô đích của khối ấy, kể cả trước khi ghép. Hành động kích hoạt Split Switch và kết thúc với hai khối đơn vị có chi phí 1, ngay cả khi một vị trí đích sau dịch chuyển là Fragile. Chi phí của SwitchCube bằng 0 vì hành động này không đổi vị trí. Quy tắc chi phí chỉ mô hình hóa mức rủi ro của Fragile, không thay đổi tính hợp lệ của nước đi.

Gọi $\pi = (a_1, \ldots, a_n)$ là lời giải. Số hành động là n, gồm cả SwitchCube; `solution_length` chỉ đếm các hành động di chuyển; `solution_cost` là $\sum_j c(s_{j-1}, a_j, s_j)$ đối với UCS/A*, còn BFS/DFS gán mỗi hành động chi phí đơn vị. Do đó, lời giải ít hành động nhất chưa chắc có ít lần di chuyển nhất hoặc có chi phí chuẩn thấp nhất.

Tóm lại, bài toán tìm kiếm được xác định bởi

$$P = (S, A, T, s_0, G, c), \tag{12}$$

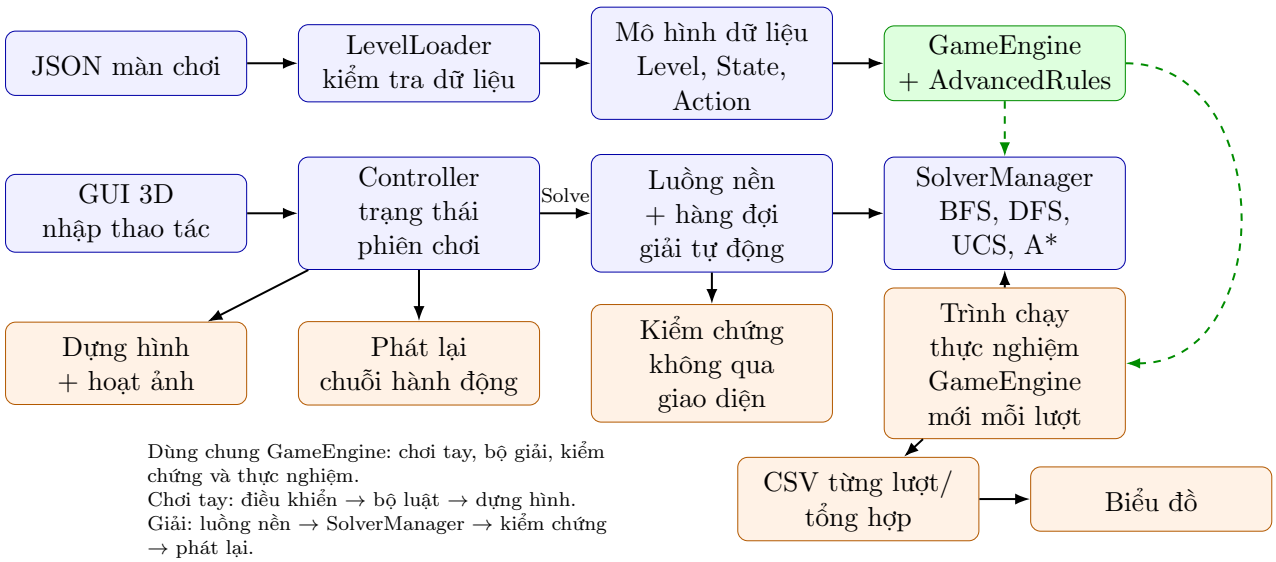
với S là tập trạng thái hợp lệ, A là tập hành động, T là quy tắc chuyển, s_0 là trạng thái đầu, G là tập đích và c là hàm chi phí chuẩn. Các chương sau cho thấy từng thành phần này được hiện thực và khai thác như thế nào.

2 Thiết kế và hiện thực hệ thống

Chương 1 đã mô tả Bloxorz như một bài toán tìm kiếm với trạng thái, hành động và quy tắc chuyển rõ ràng. Từ mô hình đó, nhóm xây dựng chương trình Python xoay quanh `GameEngine`. Thao tác của người chơi, bước tìm kiếm của bộ giải và việc kiểm chứng lời giải đều tuân theo cùng một bộ luật. Nhờ vậy, bốn thuật toán được đánh giá trên cùng một không gian trạng thái, còn giao diện và công cụ đo đạc có thể phát triển độc lập với luật chơi.

2.1 Kiến trúc tổng thể

Hình 3 cho thấy cách các thành phần phối hợp. Dữ liệu màn chơi được `LevelLoader` kiểm tra trước khi tạo đối tượng `Level`. `GameEngine` sử dụng các đối tượng dữ liệu về trạng thái, hành động và luật nâng cao để xác định kết quả của mỗi nước đi. Giao diện dùng kết quả đó để hiển thị và phát lại lời giải; các bộ giải dùng cùng bộ luật để tìm đường tới đích; còn công cụ thực nghiệm ghi nhận kết quả chạy. Trong sơ đồ, mũi tên liền biểu thị luồng xử lý hoặc dữ liệu, mũi tên nét đứt biểu thị việc dùng chung `GameEngine`.



Hình 3: Kiến trúc hệ thống và hai luồng tương tác; mọi thao tác hợp lệ được xác định bởi cùng GameEngine

Khi chơi tay, bộ điều khiển gửi hành động tới `GameEngine` và hiển thị kết quả sau hoạt ảnh. Khi giải tự động, bộ điều khiển lấy trạng thái tại thời điểm bắt đầu giải; luồng nền gọi `SolverManager` từ trạng thái đó, kiểm chứng lời giải rồi đưa kết quả vào hàng đợi. Bộ điều khiển chỉ phát lại lời giải đã được xác minh. Công cụ thực nghiệm gọi bộ giải trực tiếp, không qua giao diện hoặc luồng nền. Các nhánh cùng dùng `GameEngine` để sinh trạng thái kế tiếp và kiểm tra điều kiện đích.

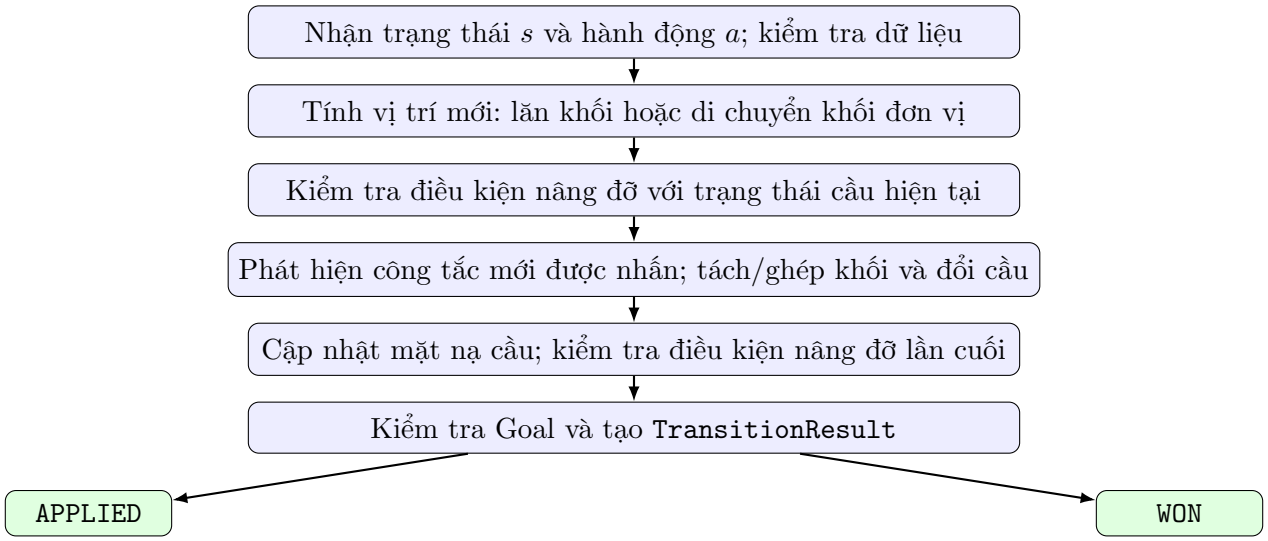
2.2 Hiện thực mô hình và GameEngine

Hệ thống đặt luật chơi trong `GameEngine` và dùng các đối tượng dữ liệu bất biến để trao đổi thông tin giữa các thành phần. `models/level.py` lưu lưới, vị trí đích, cầu và công tắc; `models/state.py` định nghĩa `GameState` cùng hai dạng khối và mặt nạ cầu; `models/enums.py` định nghĩa tư thế, hành động và loại công tắc. Tính bất biến cho phép dùng toàn bộ trạng thái làm khóa của `dict`/`set`; hai trạng thái khác mặt nạ cầu hoặc khối đang điều khiển không bị đồng nhất. `core/level_loader.py` đọc JSON, kiểm tra cấu trúc và tọa độ trước khi tạo đối tượng màn chơi. Cầu và công tắc là dữ liệu bổ sung cho lưới ô.

Bảng 7: Ánh xạ từ mô hình tìm kiếm sang hiện thực Python

Khái niệm	**Hiện thực dùng chung**
Level và luật tĩnh	`models/level.py`, `models/enums.py`, `core/level_loader.py`
S và s_0	`models/state.py`; `GameEngine.get_initial_state()`
A	Enum `Action` gồm bốn hướng và `SWITCH_CUBE`
T	`GameEngine.apply_action()`, `get_successors()` và `advanced_rules.py`
G	`GameEngine.is_goal()`
c	`solvers/costs.py: standard_step_cost()`

Mỗi hành động được `GameEngine` xử lý và trả về `TransitionResult`: mã kết quả, trạng thái kế tiếp nếu có và sự kiện cho giao diện. Với nước đi di chuyển, hệ thống tính vị trí mới, kiểm tra điều kiện nâng đỡ, xử lý hiệu ứng công tắc, tách–ghép khối và cầu, rồi kiểm tra lại trước khi xét Goal (Hình 4). Các quy tắc chi tiết nằm trong mô-đun luật nâng cao; `GameEngine` quyết định thứ tự áp dụng. Riêng SwitchCube, khi khối đã tách, chỉ đổi khối điều khiển mà không kích hoạt lại công tắc hay thay đổi cầu. Bộ giải chỉ xét các bước chuyển `APPLIED`/`WON`; nước đi không hợp lệ hoặc làm khối rơi không tạo cạnh tìm kiếm.



Hình 4: Thứ tự xử lý một hành động di chuyển trong GameEngine; nước đi rơi hoặc không hợp lệ dừng sớm

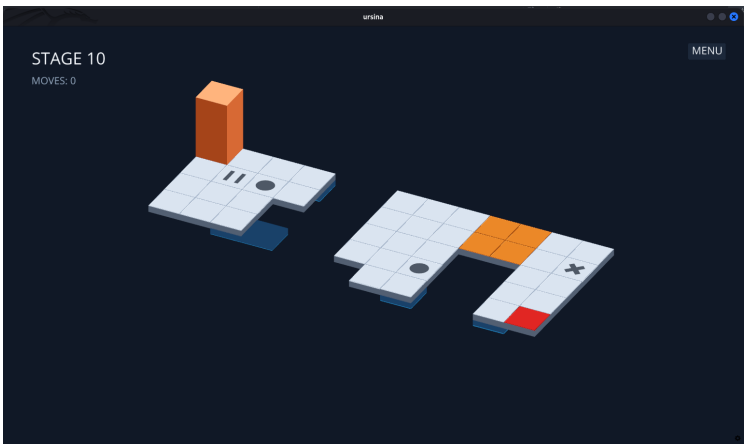
2.3 Kiến trúc hệ thống giải tự động

`SolverManager` chọn một trong bốn bộ giải rồi gọi phương thức `solve` với `GameEngine`, trạng thái đầu và các tùy chọn. Khi người dùng yêu cầu giải từ giao diện, lời gọi này chạy trong luồng nền; `SolverManager` không tự tạo luồng. `SearchOptions` cho phép cấu hình hủy tìm kiếm, giới hạn thời gian, giới hạn số nút hoặc truyền hàm chi phí và heuristic tùy chỉnh. Trong thực nghiệm, bốn bộ giải cùng sinh trạng thái kế tiếp và kiểm tra đích qua `GameEngine`. Cách quản lý tập chờ xét, thứ tự ưu tiên và điều kiện cập nhật chi phí là phần riêng của từng thuật toán ở Chương 3.

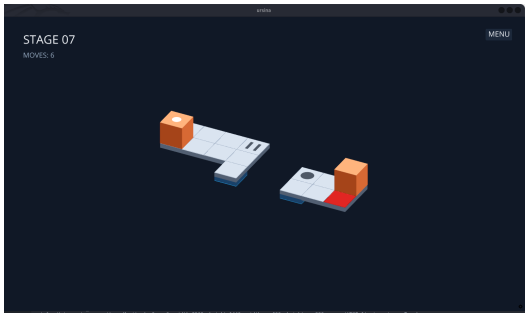
Khi tìm thấy Goal, `SolutionResult` chứa dãy hành động và `SearchMetrics`: thời gian tìm kiếm, bộ nhớ cấp phát đỉnh do Python theo dõi và số nút đã mở rộng. BFS/DFS lưu quan hệ cha–con để truy vết, UCS giữ đường đi trong mỗi phần tử hàng đợi ưu tiên, còn A* dùng `came_from`. Kết quả phân biệt `solution_length` (số hành động di chuyển), `action_count` (mọi hành động, kể cả đổi khối) và `solution_cost` (theo hàm chi phí của thuật toán). Với kết quả `FOUND`, bộ kiểm chứng áp lại từng hành động qua `GameEngine`, yêu cầu mọi bước chuyển hợp lệ và trạng thái cuối thỏa điều kiện đích. Bản thân bộ giải không phát lại lời giải.

2.4 Giao diện đồ họa và điều khiển

GUI 3D dùng Ursina trên Panda3D [4]. `gui/app.py` tạo màn hình bắt đầu, bàn chơi và menu chọn BFS/DFS/UCS/A*. Bộ dựng hình hiển thị ô, khối và hoạt ảnh theo trạng thái cùng các sự kiện của `GameEngine`.



(a) Màn chơi có công tắc, cầu, Fragile và Goal



(b) Hai khối tách ở màn 07

Hình 5: Giao diện 3D hiện có của Bloxorz Solver

Bộ điều khiển giao diện quản lý trạng thái phiên chơi và phối hợp thao tác tay, giải tự động cùng phát lại lời giải. Khi người chơi di chuyển, hành động được xử lý qua `GameEngine`; trạng thái hiển thị chỉ cập nhật sau khi hoạt ảnh kết thúc. New Game bắt đầu từ màn 01, Load Stage cho phép chọn màn khác, còn Restart đưa màn hiện tại về trạng thái đầu.

Khi người chơi chọn một thuật toán trong menu Solve, giao diện đặt lại màn hiện tại rồi chuyển trạng thái đầu cho bộ giải chạy trên luồng nền.

Luồng nền kiểm chứng lời giải tìm được trước khi gửi kết quả qua hàng đợi về bộ điều khiển. Mỗi yêu cầu giải có một mã riêng (`request_id`); kết quả cũ bị bỏ qua nếu người chơi hủy tìm kiếm, khởi động lại hoặc đổi màn. Với kết quả `FOUND`, bộ phát lại thực hiện từng hành động qua cùng `GameEngine` và đồng bộ với hoạt ảnh. Trong khi hoạt ảnh hoặc phát lại đang chạy, giao diện không nhận thêm thao tác di chuyển tay.

2.5 Đo hiệu năng và xác minh kết quả

Công cụ thực nghiệm chạy các bộ giải trực tiếp, không qua giao diện. Mỗi lượt được thực hiện trên một `GameEngine` mới và ghi kết quả vào CSV. Thời gian tìm kiếm được đo bằng `perf_counter`; bộ nhớ đỉnh là lượng cấp phát Python cao nhất do `tracemalloc` theo dõi [5]. Thời gian tải màn chơi, dựng hình, ghi CSV và phát lại không nằm trong phép đo.

Sau khi chạy, kết quả của các lượt tìm thấy lời giải được tổng hợp thành bảng CSV và biểu đồ. Bộ kiểm chứng áp lại dãy hành động qua `GameEngine` để xác nhận lời giải mà không cần mở giao diện. Tập lệnh kiểm tra báo cáo còn đối chiếu dữ liệu CSV và giải lại 60 cặp màn chơi–thuật toán để kiểm tra độc lập. Quy trình đo và kết quả được trình bày ở Chương 4 và Chương 5.

3 Các thuật toán tìm kiếm

Trên đồ thị hữu hạn của Chương 1, bốn thuật toán khác nhau ở cách quản lý tập chờ xét (*frontier*) và chọn trạng thái tiếp theo. Các mã giả dưới đây tập trung vào lõi tìm kiếm. Trạng thái đầu được kiểm tra đích ngay; các trạng thái còn lại được kiểm tra khi lấy ra khỏi tập chờ xét. Trạng thái đích không được tính vào `expanded_nodes` vì không sinh trạng thái kế tiếp. Chi phí chuẩn đã được định nghĩa ở Bảng 6; riêng BFS và DFS coi mỗi hành động là một cạnh đơn vị.

Các bảo đảm đầy đủ và tối ưu dưới đây giả định quá trình tìm kiếm không bị `CANCELLED`, `TIMEOUT` hoặc `NODE_LIMIT`. Mã giả lược bỏ các điều kiện dừng tùy chọn để tập trung vào lõi thuật toán. Các cận theo $|V|, |E|$ giả định việc sinh trạng thái kế tiếp và tra cứu bảng băm có chi phí đơn vị; khi kích thước màn chơi thay đổi, mỗi bước chuyển còn có thể duyệt danh sách công tắc và cầu.

3.1 Breadth-First Search

BFS duyệt các trạng thái theo độ sâu tăng dần bằng hàng đợi FIFO. Bản cài đặt dùng `deque` của thư viện `collections`. Từ điển `parent` vừa lưu quan hệ để truy vết lời giải, vừa đánh dấu trạng thái đã thăm. Mỗi trạng thái được đánh dấu khi đưa vào hàng đợi nên không bị thêm nhiều lần.

Algorithm 1 BFS trong hệ thống

```
1:  function BFS(engine, start)
2:      frontier ← ⟨start⟩ (FIFO)
3:      parent[start] ← ∅
4:      while frontier không rỗng do
5:          s ← popleft(frontier)
6:          if s là Goal then
7:              return RECONSTRUCT(parent, s)
8:          end if
9:          tăng expanded
10:         for all (a, t) ∈ engine.get_successors(s) do
11:             if t ∉ parent then
12:                 parent[t] ← (s, a)
13:                 append(frontier, t)
14:             end if
15:         end for
16:     end while
17:     return NOSOLUTION
18: end function
```

Trong BFS, mọi hành động có trọng số 1, kể cả SwitchCube. Do đó đường tìm được tối ưu theo `action_count`. Đại lượng `solution_length` chỉ đếm các hành động di chuyển, nên BFS không trực tiếp tối ưu đại lượng này. `solution_cost` của BFS bằng số hành động, khác chi phí chuẩn của UCS và A*.

Với đồ thị hữu hạn, BFS đầy đủ và tối ưu theo số cạnh đơn vị. Bản tìm kiếm trên đồ thị có thời gian $O(|V| + |E|)$ và bộ nhớ $O(|V|)$; theo ký hiệu cây tìm kiếm thường gặp, số nút có thể tăng theo $O(b^d)$, trong đó b là hệ số phân nhánh và d là độ sâu lời giải nông nhất.

3.2 Depth-First Search

Nhóm chọn DFS tìm kiếm trên đồ thị, không dùng tìm kiếm sâu dần (IDS). Tập chờ xét là `list` hoạt động như ngăn xếp. `GameEngine` trả các trạng thái kế tiếp theo thứ tự Up, Down, Left, Right, SwitchCube; bản cài đặt đưa chúng vào ngăn xếp theo thứ tự ngược để trạng thái đầu tiên được lấy ra trước.

Algorithm 2 DFS có chống lặp

```
1: function DFS(engine, start)
2:     frontier ← ⟨start⟩ (stack)
3:     parent[start] ← ∅
4:     while frontier không rỗng do
5:         s ← pop(frontier)
6:         if s là Goal then
7:             return RECONSTRUCT(parent, s)
8:         end if
9:         tăng expanded
10:        for all (a, t) theo thứ tự ngược của engine.get_successors(s) do
11:            if t ∉ parent then
12:                parent[t] ← (s, a)
13:                append(frontier, t)
14:            end if
15:        end for
16:    end while
17:    return NOSOLUTION
18: end function
```

Tập trạng thái đã thăm ngăn DFS lặp vô hạn khi lăn ngược hoặc đổi khối điều khiển hai lần. Trên đồ thị hữu hạn, DFS sẽ tìm thấy Goal nếu tồn tại đường đi tới đó. Lời giải đầu tiên phụ thuộc vào thứ tự trạng thái kế tiếp và không bảo đảm tối ưu số bước hay chi phí. Thời gian là $O(|V| + |E|)$; bộ nhớ của bản cài đặt vẫn có thể đạt $O(|V|)$ vì phải lưu trạng thái đã thăm và quan hệ truy vết, dù ngăn xếp DFS thường nhỏ hơn hàng đợi BFS.

3.3 Uniform-Cost Search

UCS áp dụng nguyên lý của Dijkstra trên đồ thị trạng thái [6]. Mỗi phần tử trong hàng đợi ưu tiên chứa $(g, counter, state, path)$; biến `counter` phân xử khi các giá trị ưu tiên bằng nhau mà không cần so sánh hai trạng thái. Từ điển `best_g` lưu chi phí nhỏ nhất đã biết. Phần tử có g lớn hơn giá trị hiện tại trong `best_g` đã lỗi thời, nên bị bỏ qua và không tính là một nút mở rộng.

UCS tối thiểu hóa tổng chi phí chuẩn, khác mục tiêu số hành động của BFS. Mọi chi phí ở Bảng 6 đều không âm; `best_g` chỉ cập nhật khi tìm được đường có chi phí nhỏ hơn, nên SWITCHCUBE với chi phí 0 không tạo vòng lặp cập nhật vô hạn.

Algorithm 3 Uniform-Cost Search

function UCS($engine, start, cost$)
  $frontier \leftarrow \{(0, 0, start, \langle\rangle)\}$ (min-heap)
  $bestG[start] \leftarrow 0$; $counter \leftarrow 0$
  while *frontier* không rỗng **do**
    $(g, k, s, path) \leftarrow \text{pop}(frontier)$
    if $g > bestG[s]$ **then**
      continue
    end if
    if s là Goal **then**
      return $(path, g)$
    end if
    tăng *expanded*
    for all $(a, t) \in$ `engine.get_successors(s)` **do**
      $newG \leftarrow g + cost(s, a, t)$
      if $newG < bestG.\text{get}(t, \infty)$ **then**
        $bestG[t] \leftarrow newG$
        $counter \leftarrow counter + 1$
        $path' \leftarrow path \parallel \langle a \rangle$
        $\text{push}(frontier, (newG, counter, t, path'))$
      end if
    end for
  end while
  return NoSolution
end function

Trên đồ thị hữu hạn với chi phí cạnh không âm, UCS đầy đủ và tối ưu theo tổng chi phí. Bản cài đặt sao chép chuỗi `path` khi đưa một đường tốt hơn vào hàng đợi ưu tiên. Nếu L là độ dài đường lưu lớn nhất, mỗi lần sao chép có thể tốn $O(L)$; vì vậy bộ nhớ dành cho các phần tử chờ xét có thể tới $O((|V| + |E|)L)$, còn thời gian có thêm phần sao chép bên cạnh thao tác hàng đợi ưu tiên. UCS có thể chọn thêm hành động đổi khối với chi phí 0 hoặc tránh ô Fragile có chi phí 2 để giảm tổng chi phí.

3.4 A* Search

A* chọn trạng thái theo giá trị ưu tiên

$$f(n) = g(n) + h(n), \tag{13}$$

trong đó g là chi phí từ trạng thái đầu, dùng cùng hàm chi phí với UCS, còn h ước lượng chi phí còn lại tới Goal. Heuristic *chấp nhận được* (admissible) nếu không vượt chi phí tối ưu còn lại; heuristic *nhất quán* (consistent) nếu ước lượng tại mỗi trạng thái không vượt chi phí của một cạnh cộng ước lượng ở trạng thái kế tiếp. Phạm vi áp dụng của cận dưới h được chứng minh

bên dưới.

Mỗi phần tử trong hàng đợi ưu tiên có dạng $(f, g, counter, state)$. `best_g` lưu đường rẻ nhất đã biết; khi tìm được giá trị g nhỏ hơn, A* có thể đưa cùng trạng thái vào hàng đợi lần nữa. Trạng thái đó có thể được mở rộng lại, nên số lần mở rộng có thể lớn hơn số trạng thái phân biệt $|V|$. `came_from` lưu quan hệ truy vết mà không sao chép cả đường đi vào mỗi phần tử. Cấu trúc này dựa trên A* tìm kiếm trên đồ thị của Hart, Nilsson và Raphael [7].

3.4.1 Heuristic cho khối nguyên

Gọi $C(B)$ là tâm khối: (r, c) khi đứng, $(r, c + 0{,}5)$ khi nằm ngang và $(r + 0{,}5, c)$ khi nằm dọc. Với đường đi không kích hoạt Split Switch, mỗi lần lăn dịch chuyển tâm theo khoảng cách Manhattan tối đa 1,5 và có chi phí ít nhất 1. Vì thế, tổng chi phí để tâm tới ô đích g không nhỏ hơn giá trị sau; điều kiện phải đứng trên g chỉ có thể làm số bước tăng thêm:

$$h_{normal}(B, g) = \frac{\|C(B) - g\|_1}{1{,}5}. \tag{14}$$

Đây là cận dưới cho đường đi không dịch chuyển tức thời. Một lần kích hoạt Split Switch có thể đổi vị trí khối vượt quá giới hạn dịch chuyển tâm nêu trên.

3.4.2 Heuristic cho hai khối đơn vị

Hai khối đơn vị ở P_a, P_b phải kề trực giao để có thể ghép lại trước khi khối nguyên hoàn thành màn chơi. Nếu khoảng cách giữa chúng là d, cần ít nhất $\max(0, d - 1)$ lần di chuyển một khối để đưa chúng đến vị trí ghép; đổi khối đang điều khiển có chi phí 0 nhưng không đổi vị trí. Trên đường đi không dịch chuyển tức thời, tổng khoảng cách Manhattan của hai khối tới đích giảm tối đa 3 trong một lần lăn khối nguyên và tối đa 1 trong một lần di chuyển khối đơn vị. Mỗi lần di chuyển có chi phí ít nhất 1. Cả hai đại lượng đều là cận dưới, nên ta lấy giá trị lớn hơn:

$$h_{split}(P_a, P_b, g) = \max\left(\max(0, \|P_a - P_b\|_1 - 1), \frac{\|P_a - g\|_1 + \|P_b - g\|_1}{3}\right). \tag{15}$$

Trong bản cài đặt, $h_{direct}(s, x)$ áp dụng công thức tương ứng ở trên cho khối nguyên hoặc hai khối đơn vị, với ô đích tạm x. Gọi T_q là cặp vị trí dịch chuyển của Split Switch q. Heuristic mặc định là

$$h(s) = \min\left(h_{direct}(s, g), 1 + \min_q h_{split}(T_q, g)\right), \tag{16}$$

và bỏ nhánh thứ hai nếu màn chơi không có Split Switch. Đây là cận dưới theo hàm chi phí chuẩn ở Bảng 6. Xét một đường đi bất kỳ π từ s đến Goal. Nếu không có lần dịch chuyển tức thời nào, các cận hình học ở trên cho $h_{direct}(s, g) \le \text{cost}(\pi)$. Nếu có dịch chuyển tức thời, xét

lần dịch chuyển cuối tại q. Hành động kích hoạt này có chi phí ít nhất 1; đoạn còn lại không có dịch chuyển tức thời và có chi phí ít nhất $h_{split}(T_q, g)$. Nếu hai vị trí đích kề nhau, h_{split} bằng cận theo tâm của khối vừa ghép vì Goal có tọa độ nguyên; việc ghép ngay vẫn giữ cận dưới.

Các hành động trước lần dịch chuyển tức thời cuối có chi phí không âm. Do đó $h(s) \leq \text{cost}(\pi)$ trong cả hai trường hợp, kể cả khi đường đi dùng nhiều Split Switch. Ô Void, cầu đóng và ô Fragile chỉ loại bớt đường hoặc tăng chi phí; chúng không làm mất cận dưới. Lấy đường tối ưu cho $h(s) \leq h^*(s)$ trên mọi màn chơi hợp lệ với hàm chi phí chuẩn. Lập luận này không chứng minh tính nhất quán của heuristic.

Algorithm 4 A* với cập nhật đường tốt hơn

```
1:  function AStar(engine, start, cost, h)
2:      frontier ← {(h(start), 0, 0, start)} (min-heap)
3:      bestG[start] ← 0; cameFrom ← ∅
4:      counter ← 0
5:      while frontier không rỗng do
6:          (f, g, k, s) ← pop(frontier)
7:          if g > bestG[s] then
8:              continue
9:          end if
10:         if s là Goal then
11:             return Reconstruct(cameFrom, s)
12:         end if
13:         tăng expanded
14:         for all (a, t) ∈ engine.get_successors(s) do
15:             newG ← g + cost(s, a, t)
16:             if newG < bestG.get(t, ∞) then
17:                 bestG[t] ← newG
18:                 cameFrom[t] ← (s, a)
19:                 counter ← counter + 1
20:                 f′ ← newG + h(t)
21:                 push(frontier, (f′, newG, counter, t))
22:             end if
23:         end for
24:     end while
25:     return NoSolution
26: end function
```

Khi $h = 0$, A* trở thành UCS; nhóm dùng trường hợp này làm mốc kiểm thử. Với heuristic mặc định và hàm chi phí chuẩn, A* trả về đường đi có tổng chi phí nhỏ nhất trên đồ thị hữu hạn, kể cả khi có nhiều lần dịch chuyển tức thời. Kết luận này đòi hỏi thuật toán được chạy đến khi lấy một trạng thái đích ra khỏi hàng đợi, không bị hủy, hết thời gian hoặc chạm giới hạn nút. Bảo đảm này không tự áp dụng cho hàm chi phí hoặc heuristic tùy chỉnh không thỏa điều kiện chấp nhận được. Việc mở lại trạng thái còn có thể làm tăng thời gian và bộ nhớ; hiệu quả thực tế được đánh giá ở Mục 5.

4 Thiết kế thực nghiệm

4.1 Câu hỏi đánh giá và bộ màn chơi

Thực nghiệm so sánh bốn thuật toán theo thời gian, bộ nhớ, số nút mở rộng và chất lượng lời giải. Nhóm xem heuristic có giúp A* mở rộng ít nút hơn UCS và BFS hay không, và chi phí không đều có khiến UCS chọn đường khác BFS hay không. Trường hợp DFS được xem xét theo sự đánh đổi giữa tốc độ và chất lượng lời giải. Các nhận xét chỉ dựa trên 15 màn chơi cố định, được chia theo cơ chế ở Bảng 8.

Bảng 8: Các nhóm màn chơi dùng trong thực nghiệm

Nhóm	**Màn chơi**	**Đặc trưng**
Cơ bản	01–04	Floor, Void, Goal và ba tư thế khối
Nâng cao, không tách khối	05, 06, 11, 12, 13	Fragile, cầu, Soft/Heavy Switch
Tách và ghép khối	07, 08, 14	Tách khối, đổi khối đơn vị, ghép khối và cầu
Kết hợp	09, 10, 15	Fragile, công tắc, cầu và tách/ghép khối

4.2 Môi trường thực nghiệm

Thực nghiệm được tiến hành ngày 30/10/2026 trên máy tính chạy Linux x86_64, sử dụng bộ xử lý Intel Core i7-12650H và 15 GiB RAM. Chương trình chạy bằng CPython 3.14.8 trong môi trường ảo Python. Toàn bộ thuật toán được đánh giá trên cùng máy, cùng phiên bản chương trình và cùng bộ 15 màn chơi. Cấu hình phần cứng và phần mềm được giữ như nhau giữa các lượt đo nhằm giảm sai khác do môi trường khi so sánh.

4.3 Quy trình và kiểm soát công bằng

Công cụ chạy thực nghiệm duyệt 15 màn chơi, bốn thuật toán BFS/DFS/UCS/A* và ba lần lặp cho từng cặp, tổng cộng 180 lượt. Mỗi lượt tạo một `GameEngine` mới và lấy trạng thái đầu. Các lượt được đo tuần tự để mỗi phiên tìm kiếm dùng riêng `tracemalloc`. Không đặt giới hạn thời gian hoặc số nút. Các bộ giải dùng cùng màn chơi, bộ luật, thứ tự sinh trạng thái kế tiếp và cách đo; UCS/A* dùng chung hàm chi phí chuẩn. Thời gian dựng hình và phát lại không nằm trong phép đo. CSV từng lượt lưu kết quả của mỗi lượt; CSV tổng hợp tính trung bình và độ lệch chuẩn tổng thể bằng `statistics.pstdev` trên các lượt `FOUND`.

4.4 Chỉ số đánh giá

Bốn nhóm chỉ số trong Bảng 9 tách hiệu năng tìm kiếm khỏi chất lượng đường đi. Nhóm chất lượng gồm ba đại lượng: số bước di chuyển, số hành động và tổng chi phí.

Bảng 9: Bốn nhóm chỉ số đánh giá thuật toán

Chỉ số	**Cách đo**
Thời gian tìm kiếm	Đo bằng `perf_counter`, đơn vị mili giây; không gồm tải màn chơi, dựng hình, ghi CSV hoặc phát lại
Bộ nhớ đỉnh	Lượng cấp phát Python cao nhất trừ mức ban đầu, đo bằng `tracemalloc`; không phải tổng bộ nhớ tiến trình (RSS)
Số nút mở rộng	Phần tử hợp lệ được lấy khỏi tập chờ xét và được dùng để sinh trạng thái kế tiếp; không tính trạng thái đích và phần tử lỗi thời
Chất lượng lời giải	`solution_length` đếm hành động di chuyển; `action_count` tính cả SwitchCube; `solution_cost` dùng hàm chi phí của thuật toán

4.5 Phương pháp xác minh lời giải

Bộ kiểm thử tự động đạt 203/203 trường hợp trong môi trường chạy này. Cả 180 lượt đo đều trả `FOUND`. Công cụ chạy thực nghiệm chỉ ghi chỉ số, không lưu chuỗi hành động hoặc tự phát lại trong lượt đo. Tập lệnh kiểm tra báo cáo đối chiếu 180 dòng CSV từng lượt, 60 dòng CSV tổng hợp, ba lần lặp cho mỗi cặp và các giá trị trung bình, độ lệch chuẩn. Sau đó, tập lệnh giải lại một lần cho từng cặp màn chơi–thuật toán và dùng `verify_solution` áp từng hành động qua `GameEngine` tới Goal: 60/60 lời giải hợp lệ. Thời gian xác minh độc lập không được cộng vào thời gian tìm kiếm chính thức; luồng GUI xác minh trước khi phát lại như đã trình bày ở Mục 2.

5 Kết quả và thảo luận

5.1 Bảng kết quả tổng quan

Bảng 10 tổng hợp kết quả trung bình trên 15 màn chơi. Mỗi thuật toán được chạy ba lần trên từng màn chơi.

Bảng 10: Kết quả thực nghiệm trung bình trên 15 màn chơi

Thuật toán	Hiệu năng tìm kiếm			Lời giải		
	Thời gian (ms)	Bộ nhớ (KiB)	Nút	Độ dài	Thao tác	Chi phí
BFS	73,80	174,31	619,73	15,40	16,00	16,00
DFS	53,19	143,96	438,20	125,20	139,87	139,87
UCS	96,95	173,99	671,33	15,40	16,00	16,00
A*	57,07	120,87	350,60	15,40	16,07	16,00

Trong bảng, *Độ dài* chỉ đếm các bước di chuyển, còn *Thao tác* tính cả SwitchCube. Cột *Chi phí* của BFS/DFS là số hành động đơn vị; UCS/A* dùng hàm chi phí 0/1/2 ở Bảng 6. Vì vậy, cột này chỉ cho phép so sánh trực tiếp UCS với A*. Khi đánh giá lại đường BFS bằng cùng hàm chi phí 0/1/2 của UCS, cả 15 màn chơi đều cho chi phí bằng đường UCS. Bộ màn chơi này chưa có trường hợp UCS chọn đường rẻ hơn BFS; hai thuật toán khác nhau chủ yếu ở cách duyệt và số nút mở rộng.

5.2 Biểu đồ theo màn chơi

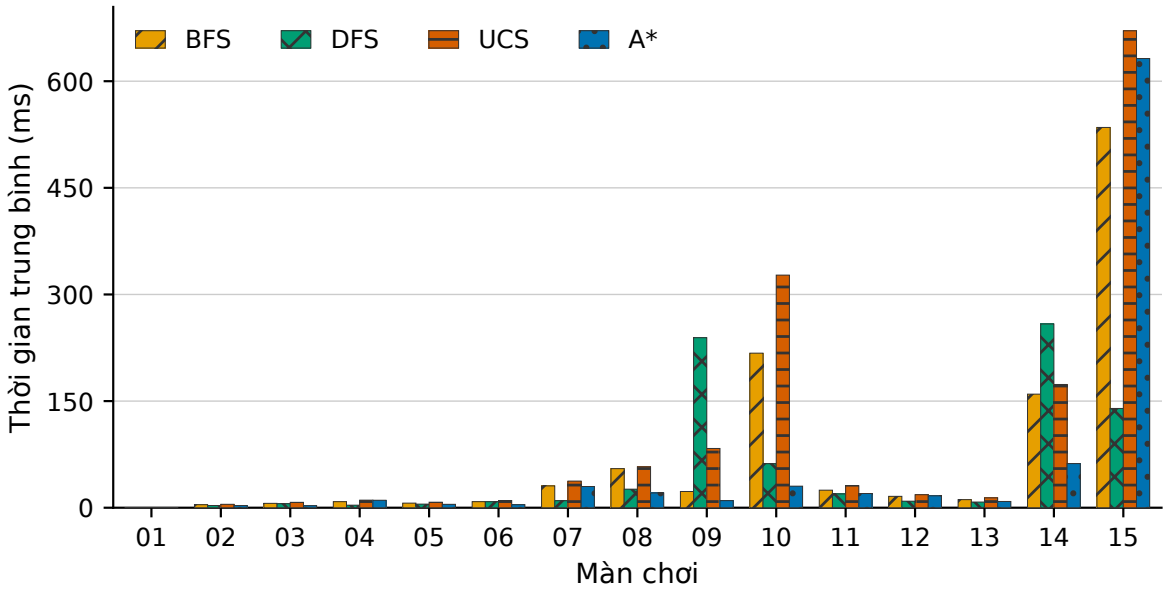
Ba biểu đồ ở Hình 6 và Hình 7 được tạo từ cùng bộ CSV bằng Matplotlib [8]. Số liệu từng màn chơi và độ lệch chuẩn thời gian được lưu trong tệp `data/benchmark_summary.csv` đi kèm báo cáo.

5.3 Chất lượng lời giải và đánh đổi

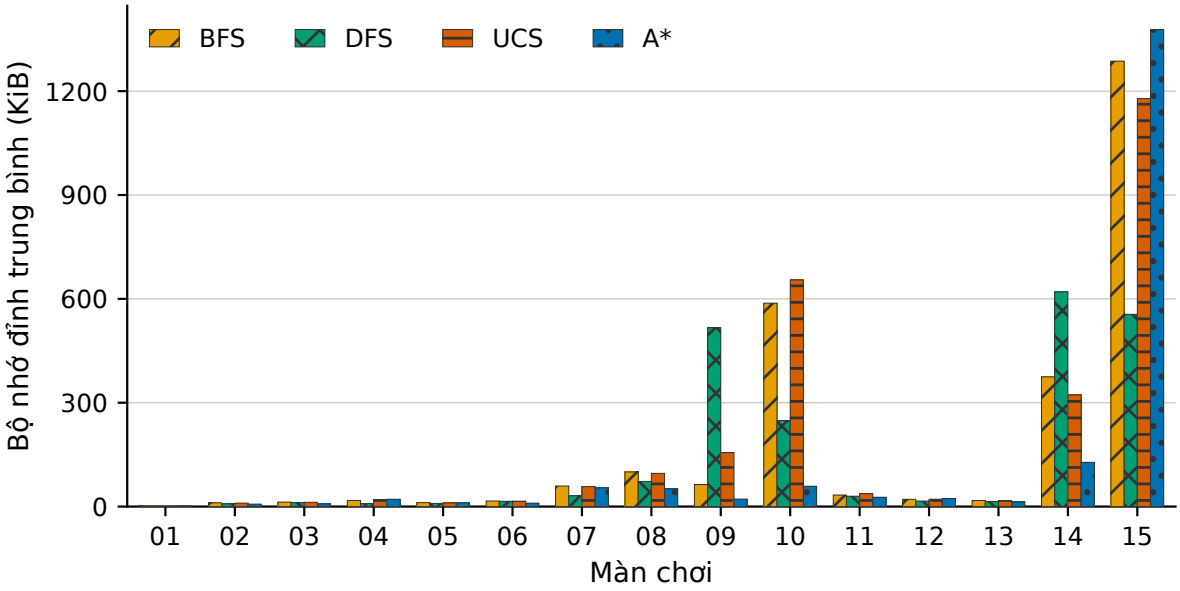
DFS có thời gian trung bình thấp nhất, 53,19 ms, nhưng lời giải dài trung bình 125,20 bước; ở màn chơi 15 con số này lên tới 724 bước. Vì thế, tốc độ của DFS phụ thuộc mạnh vào thứ tự sinh trạng thái kế tiếp và không chứng tỏ chất lượng lời giải. BFS cho lời giải ít hành động, nhưng tập chờ xét rộng và không xét chi phí của ô Fragile.

UCS tối ưu theo hàm chi phí 0/1/2 nhưng mở rộng trung bình 671,33 nút và có thời gian trung bình cao nhất trong bốn thuật toán do không có thông tin hướng đích. A* đạt cùng chi phí với UCS trên cả 15 màn chơi, đồng thời chỉ mở rộng trung bình 350,60 nút: thấp hơn BFS 43,4% và UCS 47,8%. Tại màn chơi 10, A* mở rộng 168 nút, so với 1.725 của BFS và 2.191 của UCS.

Số nút mở rộng ít hơn không bảo đảm thời gian thấp hơn. Ở màn chơi 15, A* mất 631,90 ms trong khi BFS mất 534,95 ms; thời gian tính heuristic và quản lý hàng đợi ưu tiên có thể lớn hơn phần thời gian tiết kiệm nhờ mở rộng ít nút. Trên bộ 15 màn chơi này, A* đạt sự cân bằng tốt giữa chất lượng lời giải, số nút mở rộng và bộ nhớ. UCS là mốc đối chiếu về chi phí tối ưu; BFS đơn giản nhưng dùng nhiều bộ nhớ cho tập chờ xét; DFS nhanh ở một số màn chơi nhưng chất lượng lời giải không ổn định.

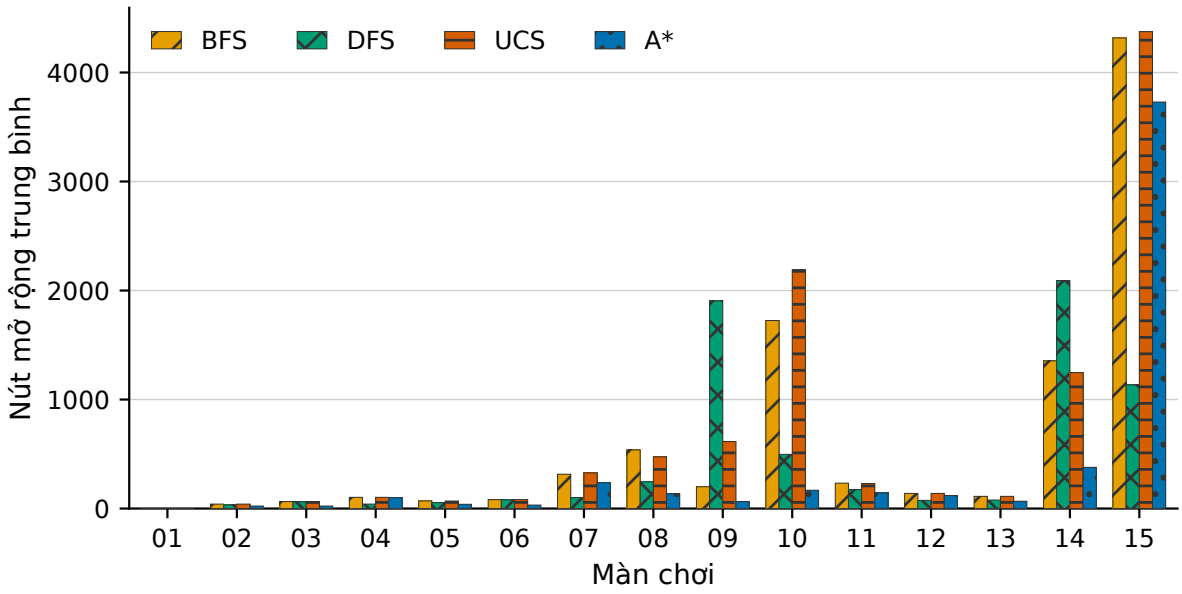


(a) Thời gian tìm kiếm trung bình



(b) Bộ nhớ Python được theo dõi, tính bằng KiB

Hình 6: Thời gian và bộ nhớ theo màn chơi và thuật toán



Hình 7: Số nút mở rộng theo màn chơi và thuật toán

Các nhận xét này chỉ dựa trên 15 màn chơi do nhóm xây dựng và ba lần lặp. Bộ nhớ đỉnh là lượng cấp phát do `tracemalloc` theo dõi, không phải tổng RAM của tiến trình; thời gian ở màn chơi nhỏ cũng có thể chịu nhiễu hệ điều hành.

6 Kết luận và hướng phát triển

6.1 Chức năng và kết quả đạt được

Đồ án đã hiện thực trò chơi Bloxorz 3D có thể điều khiển bằng tay, gồm luật cơ bản, ô Fragile, cầu, ba loại công tắc và cơ chế tách, đổi khối điều khiển, ghép khối. Bốn bộ giải BFS, DFS, UCS, A* dùng chung `GameEngine`. Giao diện hỗ trợ giải trong luồng nền, kiểm chứng và phát lại lời giải. Bộ kiểm thử đạt 203/203 trường hợp; cả 180 lượt đo hiệu năng đều tìm thấy lời giải và tập lệnh độc lập xác nhận 60/60 lời giải hợp lệ.

Trên 15 màn chơi, BFS, UCS và A* có cùng số bước di chuyển trung bình. Khi đánh giá lại bằng hàm chi phí chuẩn, đường BFS có chi phí bằng UCS trên cả 15 màn chơi. DFS có thể nhanh nhưng lời giải dài và nhạy với thứ tự sinh trạng thái kế tiếp, nổi bật là 724 bước ở màn chơi 15. A* giảm trung bình 43,4% số nút mở rộng so với BFS và 47,8% so với UCS, nhưng không luôn nhanh hơn: ở màn chơi 15, A* mất 631,90 ms so với 534,95 ms của BFS.

6.2 Giới hạn của kết luận

Heuristic A* dùng cận dưới dựa vào lần dịch chuyển tức thời cuối. Lập luận về tính chấp nhận được với hàm chi phí chuẩn nằm ở Mục 3. Bảo đảm tối ưu của A* không tự áp dụng cho hàm chi phí hoặc heuristic tùy chỉnh. Các bảo đảm tìm kiếm cũng đòi hỏi thuật toán không bị hủy, hết thời gian hoặc chạm giới hạn nút. Cận hiện hành có thể hướng dẫn tìm kiếm kém hiệu quả ở màn chơi có nhiều lần dịch chuyển tức thời.

Các nhận xét thực nghiệm chỉ dựa trên 15 màn chơi và ba lần lặp. Bộ nhớ đỉnh là lượng cấp phát Python do `tracemalloc` theo dõi, không phải tổng RAM của tiến trình; thời gian trên màn chơi nhỏ có thể chịu nhiễu hệ điều hành.

6.3 Hướng phát triển

Có thể thiết kế cận dưới mạnh hơn có xét cấu trúc kết nối của cầu và công tắc, rồi chứng minh và đo lại trước khi thay heuristic mặc định. Các thử nghiệm tiếp theo nên thêm màn chơi lớn, màn chơi sinh tự động, nhiều lần lặp với thứ tự ngẫu nhiên, đo RSS và hiển thị sai số trên biểu đồ. Về phần mềm, có thể bổ sung trình biên tập màn chơi, lưu tiến độ và hoạt ảnh phong phú hơn ở tầng giao diện.

Tài liệu tham khảo

[1] Coolmath Games, "Bloxorz." https://www.coolmathgames.com/0-bloxorz. Trang trò chơi tham khảo, truy cập ngày 27/10/2026.

[2] Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM, "CSC14003 Project 1: Bloxorz Solver," 2026. Đề bài học phần, tài liệu nội bộ của môn học.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4 ed., 2021.

[4] U. E. Contributors, "Ursina engine documentation." https://www.ursinaengine.org/documentation.html, 2026. Truy cập ngày 27/10/2026.

[5] Python Software Foundation, "Python 3.14 documentation." https://docs.python.org/3.14/, 2026. Truy cập ngày 27/10/2026.

[6] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[8] Matplotlib Development Team, "Matplotlib 3.11 documentation." https://matplotlib.org/stable/, 2026. Truy cập ngày 27/10/2026.

[9] T. Kassis, V. Agarwal, Y. He, D. Patel, and A. M. Brueckner, "Scientific agent skills: A library of procedural knowledge for research agents." https://doi.org/10.48550/arXiv.2609.00065, 2026. Bộ hướng dẫn dùng trong vòng rà soát báo cáo và biểu đồ.

A Khai báo sử dụng AI

Nhóm sử dụng OpenAI Codex để hỗ trợ đọc và đối chiếu yêu cầu đề bài với mã nguồn, đề xuất cấu trúc LaTeX, diễn đạt bản nháp tiếng Việt, chạy đo hiệu năng và kiểm tra lỗi biên dịch. Dưới đây là ba trích đoạn nguyên văn từ các yêu cầu nhóm đã gửi trong quá trình làm báo cáo:

- "Đối chiếu toàn bộ Chương 1 và Chương 2 của báo cáo với implementation thực tế."
- "Kiểm tra lập luận admissibility theo cost chuẩn 0/1/2."
- "Biên dịch PDF bằng latexmk."

Trong vòng rà soát cuối, quá trình hỗ trợ còn tham khảo các hướng dẫn *peer-review* và *scientific-visualization* thuộc bộ *Scientific Agent Skills* [9] để kiểm tra nội dung và biểu đồ.

Mọi mô tả thuật toán được đối chiếu lại với mã nguồn; số liệu được lấy từ 180 lượt đo hiệu năng thực tế và lời giải được kiểm tra bằng bộ kiểm chứng. AI không được dùng để tạo số liệu giả, xác định mức đóng góp cá nhân hoặc thay thế việc kiểm tra nội dung của nhóm. Các thành viên chịu trách nhiệm cuối cùng đối với nội dung được đưa vào báo cáo.

B Kết quả thực nghiệm đầy đủ

Bảng 11 trình bày trung bình của ba lượt. Cột Time kèm độ lệch chuẩn tổng thể; Memory là peak Python-traced memory trung bình theo KiB.

Bảng 11: Kết quả đầy đủ theo level và thuật toán

Level	Alg.	Time (ms)	Mem. (KiB)	Nodes	Length	Cost
01	A*	0.21 ± 0.01	1.9	2	2	2
01	BFS	0.20 ± 0.04	2.2	2	2	2
01	DFS	0.17 ± 0.01	1.4	2	2	2
01	UCS	0.20 ± 0.02	1.5	2	2	2
02	A*	3.00 ± 0.17	7.1	23	6	6
02	BFS	4.27 ± 0.64	10.7	41	6	6
02	DFS	3.09 ± 0.03	7.9	33	24	24
02	UCS	4.81 ± 0.21	10.0	41	6	6
03	A*	2.85 ± 0.02	7.5	23	7	7
03	BFS	6.16 ± 0.78	13.0	64	7	7
03	DFS	5.66 ± 0.03	11.6	63	37	37
03	UCS	7.59 ± 0.46	12.3	64	7	7
04	A*	10.51 ± 0.14	21.1	99	22	22
04	BFS	8.51 ± 0.14	17.7	103	22	22
04	DFS	3.59 ± 0.02	8.8	41	32	32
04	UCS	10.66 ± 0.09	19.4	103	22	22
05	A*	4.78 ± 0.16	10.7	39	11	12
05	BFS	6.48 ± 0.05	11.3	70	11	11
05	DFS	5.09 ± 0.05	9.2	55	21	21
05	UCS	7.76 ± 0.08	10.5	69	11	12
06	A*	4.26 ± 0.03	10.0	32	9	9
06	BFS	8.52 ± 0.11	16.1	82	9	9
06	DFS	8.46 ± 0.02	15.2	79	45	45
06	UCS	10.15 ± 0.17	15.6	82	9	9
07	A*	29.85 ± 0.19	54.1	236	14	14
07	BFS	30.71 ± 1.51	59.4	315	14	15
07	DFS	9.89 ± 0.02	31.2	101	62	74
07	UCS	37.46 ± 0.46	57.5	327	14	14
08	A*	20.78 ± 0.07	51.3	137	13	13
08	BFS	54.98 ± 1.98	99.9	538	13	15
08	DFS	26.11 ± 0.18	72.6	245	182	206
08	UCS	57.54 ± 0.46	95.9	474	13	13
09	A*	9.98 ± 0.11	21.6	64	10	12
09	BFS	22.84 ± 0.34	63.5	200	10	10
09	DFS	239.17 ± 2.14	516.7	1905	41	43

Tiếp ở trang sau

Bảng 11 – tiếp theo

Level	Alg.	Time (ms)	Mem. (KiB)	Nodes	Length	Cost
09	UCS	83.36 ± 0.29	155.8	615	10	12
10	A*	30.32 ± 1.55	58.7	168	14	15
10	BFS	217.45 ± 1.79	587.6	1725	14	14
10	DFS	61.94 ± 1.26	248.5	497	423	471
10	UCS	327.24 ± 2.51	654.7	2191	14	15
11	A*	20.00 ± 0.12	26.9	143	25	27
11	BFS	24.69 ± 0.95	33.4	233	25	25
11	DFS	19.40 ± 0.26	29.9	173	50	50
11	UCS	30.94 ± 0.18	37.3	229	25	27
12	A*	16.76 ± 0.04	23.4	119	19	19
12	BFS	16.02 ± 2.62	20.9	139	19	19
12	DFS	9.20 ± 1.70	16.0	75	44	44
12	UCS	18.30 ± 1.02	21.2	139	19	19
13	A*	8.81 ± 0.10	13.2	67	18	18
13	BFS	11.37 ± 0.05	17.6	112	18	18
13	DFS	8.00 ± 0.05	15.0	78	40	40
13	UCS	13.88 ± 0.18	17.0	112	18	18
14	A*	62.08 ± 0.68	127.8	378	15	15
14	BFS	159.79 ± 1.72	374.6	1355	15	17
14	DFS	258.49 ± 1.21	620.1	2091	151	171
14	UCS	173.38 ± 0.80	322.5	1247	15	15
15	A*	631.90 ± 1.15	1377.7	3729	46	49
15	BFS	534.95 ± 20.28	1286.7	4317	46	50
15	DFS	139.65 ± 0.90	555.2	1135	724	838
15	UCS	670.93 ± 22.52	1178.6	4375	46	49